SELF-DIRECTED SOFTWARE ENGINEERING

Professional curriculum specification for moving from strong Python/data scripting to production software, architecture and AI-assisted engineering judgement.

Entry point Comfortable writing non-trivial Python scripts; no need for introductory syntax instruction.	**Exit standard** Able to design, test, deploy, operate and defend the architecture of a production system.
Structure 11 modules + continuous laboratory + system-design practicum + capstone.	**Assessment model** Knowledge + unfamiliar exercises + implementation + design defence.

Design principle. This is not a collection of tutorials. Reading exists to support deliberate practice. Every module requires you to explain concepts, solve unfamiliar problems, build something, and defend engineering choices. AI is used in controlled modes so it accelerates learning rather than replacing judgement.

How to use this PDF

The document includes interactive checkboxes and completion fields. Most PDF readers support them; if yours does not, the layout is still printable. Treat completion as evidence-based: check a module only when its assessment gates have been met.

Learner:

Start date:

Capstone:

CURRICULUM SPECIFICATION - VERSION 1.0

Prepared 17 August 2026

Programme map

Modules are sequenced by prerequisite knowledge, not by calendar. Several should overlap because professional engineering skills are reinforced across contexts.

Stage	Modules	Run in parallel / dependency guidance
Foundation	M0 Tools; M1 Design; M2 Testing; M3 Algorithms; M4 Systems	Start M0. Run M1 + M2 together. Add M3 + M4 once the design/testing loop is established.
Production systems	M5 Databases; M6 Networking; M7 Delivery; M8 Security/Reliability	Run M5 + M6 together. Then M7. Start M8 once a real deployed service exists.
Advanced design	M9 Architecture; M10 Distributed Systems	Run M9 + M10 after systems foundations and production experience.
Continuous laboratory	Project I -> Project II -> Capstone	The same system evolves; avoid disconnected toy projects unless a module specifically benefits from a focused lab.

Recommended dependency flow

M0 -> (M1 + M2) -> (M3 + M4) -> (M5 + M6) -> M7 -> M8 -> (M9 + M10) -> Capstone / system-design practicum

There is intentionally no week-by-week schedule. Progress is controlled by assessment gates. If you can already demonstrate a gate convincingly, test out of that part rather than consuming material for its own sake.

Programme-level outcomes

- Convert ambiguous product or research goals into functional requirements, non-functional requirements, interfaces and acceptance criteria.
- Design maintainable codebases with explicit boundaries, dependencies and change-friendly abstractions.
- Construct a test strategy and reason about the evidence different test types provide.
- Understand enough systems, database and networking internals to diagnose performance and correctness issues instead of guessing.
- Deploy and operate a service with reproducible builds, CI/CD, observability, backups, security controls and incident practices.
- Make architecture choices through explicit quality attributes and trade-offs; document them with ADRs and diagrams.
- Reason about distributed-system failure modes and recognise when distribution is unnecessary.
- Use AI as a design challenger, implementation accelerator and reviewer while retaining human ownership of requirements, architecture and verification.

Assessment framework

Each module has four gates. Completion means all four are demonstrated, not merely read.

Gate	Question it answers	Typical evidence
1. Knowledge	Can I explain the model without leaning on notes or an LLM?	Oral explanation, diagram from memory, short written derivation.
2. Unfamiliar exercises	Can I transfer the model to a new problem?	Textbook problems, debugging task, architecture kata, tailored challenge.
3. Practical implementation	Can I make it work under realistic constraints?	Code, test suite, benchmark, deployment, failure drill.
4. Design defence	Can I justify why this solution is appropriate?	ADR, design review, whiteboard defence, trade-off memo.

AI use policy

Mode	Allowed use	Purpose
Closed-book	No AI; documentation only if the exercise permits it.	Verify retained understanding and debugging ability.
Socratic tutor	AI may ask questions, challenge assumptions and give hints, but should not provide the solution.	Expose gaps while preserving productive struggle.
Professional AI	Use agents/LLMs freely for implementation, review, alternatives, documentation and automation.	Practise Track B after you can evaluate the output.

Reusable prompt - Socratic tutor

Prompt: "Act as a senior software-engineering tutor. Do not give me the answer. Ask one question at a time that tests my mental model, points out contradictions, and forces me to justify trade-offs. If I am stuck, give the smallest useful hint. At the end, grade my understanding and list the concepts I should revisit."

Reusable prompt - Design defence

Prompt: "Act as a critical design reviewer. I will present requirements and a proposed solution. Challenge hidden assumptions, failure modes, coupling, performance, security, operability and unnecessary complexity. Do not redesign it for me until I have defended or revised each major decision."

Progress dashboard

Check a module only after passing all four gates. Use the notes/evidence fields in each module to record repository links, benchmark results, diagrams or reflections.

- [ ] M0 Engineering Tools and Professional Workflow
- [ ] M1 Software Design and Construction
- [ ] M2 Testing, Correctness and Reliability
- [ ] M3 Algorithms and Data Structures
- [ ] M4 Computer Systems
- [ ] M5 Databases and Data Systems
- [ ] M6 Networking, APIs and Web Systems
- [ ] M7 Deployment, Linux, Containers and Delivery
- [ ] M8 Security, Reliability and Observability
- [ ] M9 Software Architecture
- [ ] M10 Distributed Systems

- [ ] Project I - professional Python library complete
- [ ] Project II - deployed production service complete
- [ ] System-design practicum standard reached
- [ ] Capstone architecture, implementation and defence complete

Programme completion date:

M0 | FOUNDATION

Engineering Tools and Professional Workflow

Operate comfortably in the shell, Git, environments, debugging, profiling, packaging and reproducible workflows.

Parallel / prerequisite guidance: Begin first. Continue as a running skills strand throughout M1-M8.

Learning outcomes

- Shell fluency and composable command-line tools
- Git beyond add/commit: branching, rebase, bisect, reflog and conflict resolution
- Python packaging with pyproject.toml and dependency locking
- Debuggers and profilers
- Environment variables, permissions and processes
- Reproducible local developer environments

Reading specification

Resource	Required scope	Priority
MIT Missing Semester 2026	Complete the full 2026 sequence: shell; command-line environment; development environment/tools; debugging/profiling; Git; packaging/shipping; agentic coding; beyond the code; code quality.	Core
Software Engineering at Google	Ch. 22 Version Control and Branch Management; Ch. 24 Build Systems and Build Philosophy; Ch. 27 Dependency Management.	Selective

Further reading

- Software Engineering at Google, Ch. 23 Code Search and Ch. 26 Static Analysis.

Practice and exercises

- Reproduce a clean project environment on a second machine or disposable VM from only the repository instructions.
- Use git bisect to locate an intentionally introduced regression.
- Profile a slow Python script and produce a one-page bottleneck report before optimising it.
- Package one existing script as an installable command-line application.

Module project / laboratory

Engineering bootstrap: create a professional repository template with src/ layout, pyproject.toml, formatting/linting/type-check commands, test command, pre-commit hooks, README setup instructions and a CI skeleton.

Assessment gates

- [ ] Gate 1: Explain what PATH, a process, an environment variable, a virtual environment and a package are without notes.
- [ ] Gate 2: Recover from a deliberately messy Git state without deleting the repository.
- [ ] Gate 3: Package and install your project into a clean environment.
- [ ] Gate 4: Demonstrate a profiler-guided improvement and explain why the profiler evidence supports the change.

M1 | FOUNDATION

Software Design and Construction

Learn to control complexity through abstraction, information hiding, modularity, interfaces and disciplined refactoring.

Parallel / prerequisite guidance: Run in parallel with M2. M0 remains active.

Learning outcomes

- Managing complexity as the central design problem
- Abstraction, deep modules and information hiding
- Cohesion, coupling and dependency direction
- Interface design and error semantics
- Separation of concerns and state management
- Code smells and behaviour-preserving refactoring
- Documentation as part of design
- Design alternatives and explicit trade-offs

Reading specification

Resource	Required scope	Priority
A Philosophy of Software Design, 2nd ed.	Core: Ch. 1-11 (complexity, deep modules, information hiding, layering, pulling complexity down, decomposition, errors, design-it-twice). Also Ch. 14-18 (names, comments-first design, modifying code, consistency, obvious code), Ch. 20 Designing for Performance, Ch. 21 Decide What Matters.	Core
Refactoring, 2nd ed.	Ch. 1 First Example; Ch. 2 Principles; Ch. 3 Bad Smells; Ch. 4 Building Tests. Then use Ch. 6-11 as a working catalog while refactoring.	Core/Reference
Software Engineering at Google	Ch. 1 What Is Software Engineering?; Ch. 14 Code Review; Ch. 15 Documentation.	Core

Further reading

- A Philosophy of Software Design Ch. 12-13 on documentation in depth and Ch. 19 Software Trends.
- Refactoring Ch. 12 Dealing with Inheritance.
- Software Engineering at Google Ch. 6-7 on teamwork and knowledge sharing.

Practice and exercises

- Take a 300-800 line script and identify responsibilities, hidden dependencies, state, interfaces and change hotspots before touching code.
- Design two materially different decompositions for the same subsystem, then compare them using complexity, coupling and information leakage.
- Refactor a deliberately poor service while preserving behaviour through tests.
- Write a design note for a new feature before implementation, including public interfaces and failure semantics.

Module project / laboratory

Project I begins: refactor an existing backtesting/research script into a library with explicit domain modules, typed public interfaces and no UI assumptions.

Assessment gates

- [ ] Gate 1: Given unfamiliar code, identify at least five concrete sources of complexity and rank them.
- [ ] Gate 2: Defend module boundaries and dependency direction on a whiteboard.
- [ ] Gate 3: Refactor a feature without changing observable behaviour.
- [ ] Gate 4: Explain at least one design choice you would now reverse and why.

M2 | FOUNDATION

Testing, Correctness and Reliability

Design test strategies that give evidence of behaviour, not merely code coverage.

Parallel / prerequisite guidance: Run in parallel with M1; continue through every later project.

Learning outcomes

- Test scope: unit, integration, system and end-to-end
- Behaviour vs implementation testing
- Fixtures, fakes, stubs, mocks and seams
- Property-based testing and invariants
- Regression tests and deterministic reproducibility
- Coverage as a diagnostic, not a quality target
- CI and test isolation
- Testability as a design property

Reading specification

Resource	Required scope	Priority
Python Testing with pytest, 2nd ed.	Read: Getting Started; Writing Test Functions; Fixtures; Builtin Fixtures; Parametrization; Markers; Strategy; Configuration Files; Coverage; Mocking; tox and Continuous Integration; Testing Scripts and Applications; Debugging Test Failures.	Core
Software Engineering at Google	Ch. 16 Testing Overview; Ch. 17 Unit Testing; Ch. 18 Test Doubles; Ch. 19 Larger Testing.	Core
Hypothesis documentation	Core concepts, strategies, property-based testing, stateful testing and shrinking.	Core

Further reading

- Python Testing with pytest: Third-Party Plugins, Building Plugins and Advanced Parametrization.
- Software Engineering at Google Ch. 29 Continuous Integration.
- Mutation testing with mutmut or equivalent after the main test suite is mature.

Practice and exercises

- Write an explicit test strategy for the backtester before adding tests.
- Identify ten domain invariants and encode at least five with Hypothesis.
- Create a failure-injection test for partial job execution.
- Replace an over-mocked unit test with a higher-fidelity test and explain the trade-off.

Module project / laboratory

Build a layered test suite for Project I: unit, integration and property tests; deterministic fixtures; CI; coverage report; regression fixtures for previously fixed bugs.

Assessment gates

- [ ] Gate 1: Explain why a specific test belongs at its chosen scope.
- [ ] Gate 2: Demonstrate a property test finding a case you did not hand-write.
- [ ] Gate 3: Show that the suite fails for a deliberately seeded bug.
- [ ] Gate 4: Defend where you used mocks and where you deliberately avoided them.

M3 | FOUNDATION

Algorithms and Data Structures

Build enough algorithmic literacy to choose data structures and reason about time/space trade-offs.

Parallel / prerequisite guidance: Can run alongside late M1/M2 and early M4. Do not let it become a separate interview-grind track.

Learning outcomes

- Asymptotic time and space analysis
- Arrays, lists, stacks, queues, hash tables
- Trees, heaps and graphs
- Sorting and searching
- Recursion and divide-and-conquer
- Dynamic programming and greedy methods
- Amortised analysis
- Graph traversal and shortest paths

Reading specification

Resource	Required scope	Priority
Introduction to Algorithms, 4th ed. (CLRS)	Core: Ch. 1-4; Ch. 6-7; Ch. 10-12; Ch. 15-17; Ch. 22-24. Focus on modelling, asymptotic reasoning and choosing structures rather than memorising proofs.	Core/Selective

Further reading

- CLRS Ch. 5 probabilistic analysis; Ch. 8-9 sorting/order statistics; Ch. 13-14 balanced/augmented trees; Ch. 21 disjoint sets; Ch. 25-26 all-pairs shortest paths and max flow; advanced parts as needed.

Practice and exercises

- For each major data structure, implement a small version once and then use standard-library equivalents professionally.
- Solve a curated set of ~40-60 problems spanning the core structures; write a short explanation of why each structure/algorithm was selected.
- Analyse one performance problem from your own code before benchmarking it.
- Compare two solutions on both asymptotic complexity and actual benchmark behaviour.

Module project / laboratory

Add a benchmark notebook/report to Project I analysing one algorithmic bottleneck and at least two alternative implementations.

Assessment gates

- [ ] Gate 1: Derive Big-O for unfamiliar code.
- [ ] Gate 2: Select a data structure from requirements and defend the choice.
- [ ] Gate 3: Recognise when asymptotic improvement will not matter at the expected scale.
- [ ] Gate 4: Explain a dynamic-programming solution without relying on a memorised template.

M4 | FOUNDATION

Computer Systems

Understand what programs are doing beneath Python: machine code, memory, processes, virtual memory, I/O and concurrency.

Parallel / prerequisite guidance: Run with M3; complete before serious distributed-systems work.

Learning outcomes

- Representation and machine-level execution
- Compilation, linking and binaries
- Memory hierarchy, locality, stack/heap and allocation
- Processes, signals and exceptional control flow
- Virtual memory
- Files and system calls
- Sockets
- Threads, race conditions, locks and deadlocks
- Performance profiling

Reading specification

Resource	Required scope	Priority
Computer Systems: A Programmer's Perspective, 3rd ed.	Read Ch. 1 Tour; Ch. 2 Representation; Ch. 3 Machine-Level Programs; Ch. 5 Optimising Program Performance; Ch. 6 Memory Hierarchy; Ch. 7 Linking; Ch. 8 Exceptional Control Flow; Ch. 9 Virtual Memory; Ch. 10 System-Level I/O; Ch. 11 Network Programming; Ch. 12 Concurrent Programming.	Core
C language support	Use K&R; or the CS:APP C/Unix tutorials only as needed to become comfortable enough for systems labs.	Support

Further reading

- CS:APP Ch. 4 Processor Architecture for deeper hardware detail.
- CMU 15-213/15-513 lecture material and labs where publicly available.

Practice and exercises

- Write a dynamic array in C with correct allocation/freeing.
- Inspect compiler-generated assembly for a small C function at different optimisation levels.
- Create and fix a race condition.
- Use strace/lsof/gdb (or platform equivalents) to diagnose a small failing program.
- Benchmark cache-friendly vs cache-hostile memory access.

Module project / laboratory

Systems lab portfolio: 4-6 small C/Linux labs plus a short write-up connecting each concept back to Python behaviour and performance.

Assessment gates

- [ ] Gate 1: Explain stack vs heap, process vs thread, virtual vs physical memory and file descriptor without notes.
- [ ] Gate 2: Diagnose a small native crash from evidence.
- [ ] Gate 3: Explain why two logically equivalent loops differ in performance.
- [ ] Gate 4: Demonstrate a concurrency bug and the mechanism that fixes it.

M5 | PRODUCTION SYSTEMS

Databases and Data Systems

Design relational data models and understand storage, indexes, query execution, transactions, concurrency and recovery.

Parallel / prerequisite guidance: Run with M6. Start after M1/M2 and enough M4 to understand I/O, memory and concurrency.

Learning outcomes

- Relational modelling and normalisation
- Constraints and schema evolution
- Indexes and B-trees/LSM concepts
- Query plans and optimisation
- Transactions and ACID
- Isolation levels, MVCC and locking
- Concurrency anomalies
- Recovery and durability
- Choosing relational vs document/other models

Reading specification

Resource	Required scope	Priority
CMU 15-445/645 Intro to Database Systems	Core lecture sequence on relational model/SQL, storage, buffer management, indexes, query processing/optimisation, transactions/concurrency and recovery. Treat projects as optional advanced labs rather than mandatory full replication.	Core Course
Designing Data-Intensive Applications, 2nd ed.	Ch. 1 Trade-Offs in Data Systems Architecture; Ch. 2 Nonfunctional Requirements; Ch. 3 Data Models and Query Languages; Ch. 4 Storage and Retrieval; Ch. 8 Transactions.	Core

Further reading

- DDIA Ch. 5 Encoding and Evolution.
- DDIA Ch. 6-7 replication and sharding as a bridge into M10.
- Selected CMU 15-445 internal implementation projects if you want deeper DBMS internals.

Practice and exercises

- Design an experiment-management schema from written requirements; include constraints and indexes.
- Use EXPLAIN/EXPLAIN ANALYZE to diagnose and improve real PostgreSQL queries.
- Reproduce at least two isolation anomalies with concurrent sessions.
- Write migration up/down plans for a non-trivial schema change.

Module project / laboratory

Add PostgreSQL persistence to Project I for datasets, strategies, runs, parameters and results. Include migrations, constraints, transaction boundaries and query-performance evidence.

Assessment gates

- [ ] Gate 1: Draw the schema and justify every major relationship/index.
- [ ] Gate 2: Explain MVCC and at least three isolation anomalies.
- [ ] Gate 3: Given a slow query, use the query plan rather than guessing.
- [ ] Gate 4: Defend PostgreSQL against at least two plausible alternatives for this project.

M6 | PRODUCTION SYSTEMS

Networking, APIs and Web Systems

Understand the path from browser to service and design robust network-facing APIs.

Parallel / prerequisite guidance: Run with M5; feed directly into M7.

Learning outcomes

- Layered networking model and packet flow
- DNS, IP, TCP, UDP and ports
- HTTP semantics, caching and status codes
- TLS concepts
- Sockets and connection lifecycles
- REST and RPC trade-offs
- API contracts, versioning and idempotency
- Authentication/session concepts
- Latency, bandwidth, timeouts and retries

Reading specification

Resource	Required scope	Priority
Computer Networking: A Top-Down Approach, 9th ed.	Core: Ch. 1 foundations; Ch. 2 Application Layer (especially HTTP, DNS, sockets); Ch. 3 Transport Layer (TCP/UDP, reliability, congestion); Ch. 4 sections 4.1-4.3 on forwarding/IP.	Core
DDIA, 2nd ed.	Ch. 5 sections on dataflow through services: REST/RPC and event-driven dataflow.	Selective

Further reading

- Kurose/Ross Ch. 5 routing/control plane; Ch. 6 link layer and data-centre networking; security chapter/sections as support for M8.

Practice and exercises

- Use curl and browser dev tools to trace an HTTP exchange.
- Capture and interpret a TCP connection with Wireshark.
- Design an API on paper before implementing it; specify failure semantics and idempotency.
- Implement a tiny TCP client/server once to remove the magic from sockets.

Module project / laboratory

Expose Project I via a FastAPI service. Define the OpenAPI contract first. Add request validation, stable error models, idempotent job submission and API-level integration tests.

Assessment gates

- [ ] Gate 1: Explain what happens from entering a domain name to receiving a response.
- [ ] Gate 2: Explain why TCP retries do not make an application operation idempotent.
- [ ] Gate 3: Defend endpoint methods/status codes and error semantics.
- [ ] Gate 4: Diagnose a deliberately introduced timeout/connection problem.

M7 | PRODUCTION SYSTEMS

Deployment, Linux, Containers and Delivery

Turn localhost software into a repeatably built, configured, deployed and recoverable service.

Parallel / prerequisite guidance: Follow M5/M6. Continue M2 testing practices and M0 tooling.

Learning outcomes

- Linux service operation
- Images, containers, volumes and networks
- Docker Compose
- Configuration vs code and secrets
- CI/CD pipelines
- DNS/TLS/reverse proxies
- Managed databases and object storage
- Deployment strategies and rollback
- Backups and restoration
- Infrastructure cost awareness

Reading specification

Resource	Required scope	Priority
MIT Missing Semester 2026	Revisit Packaging and Shipping Code; development environment/tools; debugging/profiling; beyond-the-code and code-quality sessions in the context of deployment.	Core
Software Engineering at Google	Ch. 29 Continuous Integration; Ch. 30 Continuous Delivery; Ch. 31 Compute as a Service.	Core

Further reading

- Software Engineering at Google Ch. 24 Build Systems, if not completed in M0.
- Google SRE book Ch. 8 Release Engineering.
- Container/runtime documentation for Docker and your chosen cloud provider.

Practice and exercises

- Containerise the API and database locally with Compose.
- Deploy to a fresh environment from repository instructions only.
- Break configuration deliberately and diagnose it from logs/environment inspection.
- Perform a rollback and a database restore drill.

Module project / laboratory

Project II: deploy the application publicly or to a private cloud environment using HTTPS, containers, managed persistence where appropriate and automated CI/CD.

Assessment gates

- [ ] Gate 1: Rebuild and redeploy from zero without undocumented local steps.
- [ ] Gate 2: Explain the runtime path from DNS to container to database.
- [ ] Gate 3: Demonstrate rollback and recovery.
- [ ] Gate 4: Defend what you did not introduce (for example Kubernetes) as clearly as what you did.

M8 | PRODUCTION SYSTEMS

Security, Reliability and Observability

Build systems that can be defended, monitored, diagnosed and recovered.

Parallel / prerequisite guidance: Begin once an actual deployed service exists; continue forever as an engineering discipline.

Learning outcomes

- Threat modelling and trust boundaries
- Authentication/authorisation, secrets and least privilege
- Injection, XSS, CSRF, SSRF and supply-chain risks
- Structured logging, metrics and tracing
- SLIs, SLOs and error budgets
- Health checks and alerting
- Incident response and postmortems
- Backups, disaster recovery and load shedding
- Failure injection and graceful degradation

Reading specification

Resource	Required scope	Priority
OWASP Top 10:2025	Read all categories as an application-security awareness baseline.	Core
OWASP ASVS 5.0	Use the standard as a verification checklist; focus first on Level 1 requirements relevant to your service and then selected Level 2 areas.	Core Reference
Google Site Reliability Engineering	Ch. 3 Embracing Risk; Ch. 4 Service Level Objectives; Ch. 6 Monitoring Distributed Systems; Ch. 8 Release Engineering; Ch. 9 Simplicity.	Core
Site Reliability Workbook	Ch. 2 Implementing SLOs; Ch. 4 Monitoring; Ch. 5 Alerting on SLOs; Ch. 9 Incident Response; Ch. 10 Postmortem Culture; Ch. 11 Managing Load.	Core/Practical

Further reading

- OWASP Web Security Testing Guide for targeted test cases.
- Google Building Secure & Reliable Systems as advanced reading.

Practice and exercises

- Threat-model Project II: assets, actors, entry points, trust boundaries, threats and mitigations.
- Map your service against relevant ASVS controls and record gaps.
- Define three SLIs and a realistic SLO for the service.
- Run failure drills: database unavailable, worker crash, disk full, dependency timeout, duplicate request.
- Write a blameless postmortem for one induced incident.

Module project / laboratory

Production hardening milestone: security checklist, threat model, monitoring dashboard, actionable alerts, backup/restore evidence, incident runbook and SLO document.

Assessment gates

- [ ] Gate 1: Explain the difference between authentication and authorisation and identify the enforcement points in your system.
- [ ] Gate 2: Show monitoring that would detect a real failure mode.
- [ ] Gate 3: Demonstrate one recovery procedure.
- [ ] Gate 4: Prioritise security/reliability risks by impact and likelihood rather than by checklist anxiety.

M9 | ADVANCED DESIGN

Software Architecture

Make and defend high-level system choices through quality attributes, styles, boundaries, ADRs and trade-off analysis.

Parallel / prerequisite guidance: Begin after you have built and operated a non-trivial system. Continue alongside M10 and the capstone.

Learning outcomes

- Architecture vs design
- Architectural characteristics / quality attributes
- Modularity, components and granularity
- Monolith vs distributed architecture
- Layered, modular-monolith, service-based, event-driven and microservices styles
- C4/UML-style architecture communication
- ADRs and governance
- Risk storming
- Fitness functions
- Architecture as trade-off analysis

Reading specification

Resource	Required scope	Priority
Fundamentals of Software Architecture, 2nd ed.	Core: Ch. 1-9; Ch. 10 Layered; Ch. 11 Modular Monolith; Ch. 14 Service-Based; Ch. 15 Event-Driven; Ch. 18 Microservices; Ch. 19 Choosing a Style; Ch. 20 Patterns; Ch. 21 Architectural Decisions; Ch. 22 Risk; Ch. 23 Diagramming; Ch. 26 Architectural Intersections; Ch. 27 Laws Revisited.	Core
DDIA, 2nd ed.	Re-read Ch. 1-2 specifically as architecture trade-off/nonfunctional-requirement material.	Core Reinforcement

Further reading

- Fundamentals Ch. 12 Pipeline, Ch. 13 Microkernel, Ch. 16 Space-Based and Ch. 17 Orchestration-Driven SOA.
- Fundamentals Ch. 24-25 for team/leadership skills.
- Software Architecture in Practice, 4th ed., especially quality attributes and quality-attribute scenarios.

Practice and exercises

- Do one architecture kata per major style; force yourself to name the quality attributes that drive the choice.
- Write ADRs for database, caching, job execution and deployment choices in your current project.
- Redesign the same system under three different constraints: tiny team/low cost, high availability, and very high throughput.
- Run a risk-storming exercise on your deployed system.

Module project / laboratory

Architecture review of Project II: produce context/container/component diagrams, quality-attribute scenarios, ADR set, risk register and a proposed redesign - then explicitly reject unnecessary complexity.

Assessment gates

- [ ] Gate 1: Given requirements, identify the top architectural drivers before naming technologies.
- [ ] Gate 2: Defend a modular monolith against microservices, and then argue the opposite case under changed constraints.
- [ ] Gate 3: Explain the consequences of at least five ADRs.
- [ ] Gate 4: Whiteboard a system and handle interviewer changes without losing consistency.

M10 | ADVANCED DESIGN

Distributed Systems

Reason about partial failure, replication, consensus, consistency, partitioning and distributed coordination.

Parallel / prerequisite guidance: Run after M4 and with M9. This is the most advanced module.

Learning outcomes

- Partial failure and unreliable networks
- Replication and failover
- Partitioning/sharding
- Consistency models and linearizability
- Logical clocks and time
- Consensus and Raft
- Distributed transactions and idempotency
- Queues, at-least-once delivery and deduplication
- Backpressure and streaming
- Distributed locks/leases and their failure modes

Reading specification

Resource	Required scope	Priority
DDIA, 2nd ed.	Core: Ch. 6 Replication; Ch. 7 Sharding; Ch. 8 Transactions (review); Ch. 9 Trouble with Distributed Systems; Ch. 10 Consistency and Consensus; Ch. 11 Batch Processing; Ch. 12 Stream Processing; Ch. 13 Philosophy of Streaming Systems.	Core
MIT 6.5840 Distributed Systems, Spring 2026	Use lectures/readings and selected labs: MapReduce; Key/Value server; Raft; fault-tolerant KV service. Treat the full course as an advanced optional challenge if Go and time permit.	Core/Advanced Labs

Further reading

- MIT 6.5840 additional papers and final-project ideas.
- DDIA Ch. 14 Doing the Right Thing for responsibility, privacy and societal constraints.

Practice and exercises

- Implement a small replicated state machine or complete selected MIT labs.
- Design an idempotent job-processing protocol under duplicate delivery.
- Reason through a network partition and state what each node can know.
- Compare consistency requirements for trading orders, analytics results and cached market data.

Module project / laboratory

Distributed extension: add a worker/job model to the research platform only if measurements or product requirements justify it. Document exactly why distribution is needed and which new failure modes it creates.

Assessment gates

- [ ] Gate 1: Explain why partial failure is fundamentally different from local failure.
- [ ] Gate 2: Explain linearizability and consensus with a concrete example.
- [ ] Gate 3: Design retry/idempotency behaviour for a distributed workflow.
- [ ] Gate 4: Identify when a single-node or single-database design is superior.

Continuous laboratory specification

The laboratory is deliberately cumulative. You should feel the consequences of earlier design decisions as the system acquires persistence, an API, deployment, monitoring and eventually architectural pressures.

Milestone	Minimum deliverables	What it proves
Project I - Professional library	src/ package; pyproject; typed API; configuration; structured logging; unit/integration/property tests; CI; benchmarks; documentation.	You can turn research code into maintainable, testable software independent of a UI.
Project II - Production service	FastAPI or equivalent; PostgreSQL; migrations; Docker/Compose; HTTPS deployment; CI/CD; secrets/config; monitoring; backup/restore; threat model.	You can cross the localhost-to-production boundary.
Architecture review	Context/container/component diagrams; NFRs; ADRs; risk register; SLOs; alternatives rejected.	You can explain the system and defend its structure.
Capstone - Research platform	A justified architecture for ingestion, data/versioning, experiments, backtests, optimisation/ML jobs and results. Distribution only where evidence requires it.	You can integrate engineering and domain knowledge into a defensible system.

- [ ] Project I accepted
- [ ] Project II accepted
- [ ] Architecture review accepted
- [ ] Capstone accepted

Engineering notebook requirement

For meaningful decisions, record: problem; initial intuition; constraints; options; decision; why; evidence; what you got wrong; what changed after implementation; and what you would choose now. Promote durable decisions into ADRs inside the repository.

System-design practicum

Begin during M9 and continue through M10. The goal is not memorising canonical architectures. The goal is a repeatable reasoning process under changing constraints.

Interview sequence

Level	Suggested systems	Focus
1 - Foundations	URL shortener; paste service; rate limiter	Requirements, API, data model, simple capacity estimates.
2 - Components	Job queue; notifications; autocomplete; file storage	Asynchrony, retries, caching, durability, failure handling.
3 - Large systems	Dropbox; video platform; ride-hailing; social feed	Partitioning, replication, consistency, hot spots, operability.
4 - Domain capstone	Trading exchange; distributed backtester; feature store; market-data platform	Domain invariants, latency, correctness, auditability, scale and failure.

Whiteboard template

- [ ] 1. Clarify users and functional requirements
- [ ] 2. Define non-functional requirements / quality attributes
- [ ] 3. Estimate scale only where it changes design
- [ ] 4. Define external APIs and key data model
- [ ] 5. Draw a simple high-level architecture
- [ ] 6. Identify the critical path and state ownership
- [ ] 7. Reason about failures, retries and idempotency
- [ ] 8. Find bottlenecks and scaling limits
- [ ] 9. Address security, observability and operations
- [ ] 10. State alternatives, trade-offs and what would trigger redesign

Reusable interview prompt

Prompt: "Run a system-design interview appropriate to my current curriculum level. Give me only the initial problem. Stay in interviewer role. Answer clarification questions but do not volunteer architecture. Challenge weak assumptions and change one requirement midway through. At the end, score requirements gathering, data model, interfaces, architecture, scalability, reliability, security, operability and trade-off reasoning separately."

Exercise generator and oral-exam prompts

Use these to generate fresh practice rather than repeatedly solving familiar examples. Replace the bracketed module or system with the relevant topic.

Debugging lab

"Create a self-contained broken repository exercise for [module]. Include 3-5 interacting faults. Give me the repository/files but not the diagnosis. I must use evidence to find the root causes. Grade my debugging process, not just whether I fixed it."

Transfer problem

"Give me an unfamiliar engineering scenario testing [concept]. Do not use the same domain as the textbook. Require me to choose between at least three plausible approaches and defend the trade-offs."

Code review

"Give me a 150-300 line code sample with realistic design, correctness and testability issues. Do not label them. Act as the author in a code review and make me justify each requested change."

Architecture kata

"Give me a compact product brief with 6-10 requirements and 3 hidden architectural drivers. I must ask questions to uncover them. Do not reveal the drivers unless my questions reach them."

Failure drill

"Take my described architecture and inject one realistic failure at a time. Ask what users observe, what state may be inconsistent, what telemetry fires and how I recover. Escalate only after I handle the current failure."

Oral examination

"Give me a 20-question oral exam for [module], adaptive in difficulty. Ask one question at a time, follow up on vague answers, and score conceptual accuracy, depth, transfer and confidence calibration."

Master reading plan

This table is the programme-level view. "Core" means read deliberately and take notes. "Selective" means read the specified chapters/sections for the module. "Reference" means keep nearby and use during practical work.

Resource	Core use	Do not over-read initially
A Philosophy of Software Design, 2nd ed.	M1: Ch. 1-11, 14-18, 20-21	Ch. 12-13 and 19 can follow after you have a codebase to critique.
Refactoring, 2nd ed.	M1: Ch. 1-4; use Ch. 6-11 as catalog	Do not memorise every refactoring; apply them to real code.
Software Engineering at Google	Ch. 1, 14-19, 22, 24, 27, 29-31 across modules	Culture/leadership/tooling chapters are excellent but can be deferred until context exists.
Python Testing with pytest, 2nd ed.	Core mechanics + strategy + coverage + mocking + CI + debugging	Plugin authoring and advanced parametrisation are optional until needed.
CLRS, 4th ed.	Ch. 1-4, 6-7, 10-12, 15-17, 22-24	Treat the rest as reference/further study, not a prerequisite to shipping software.
CS:APP, 3rd ed.	Ch. 1-3, 5-12	Ch. 4 processor architecture is useful but not essential for the initial objective.
Kurose/Ross, 9th ed.	Ch. 1-3 + 4.1-4.3	Routing/link/wireless depth can come later unless your work demands it.
DDIA, 2nd ed.	M5: Ch. 1-4,8; M10: Ch. 6-13	Do not read it all at the start; later chapters become far more useful after production experience.
Fundamentals of Software Architecture, 2nd ed.	M9 core chapter set	Read specialised architecture styles after understanding their drivers.
Google SRE + Workbook	Selected SLO, monitoring, release, incident/load chapters	Use as operational guidance, not as a demand to imitate Google scale.

Source notes and curriculum provenance

The reading map was cross-checked against current publisher/author contents pages and 2026 university course pages where available. University courses are used as structural inspiration, not represented as enrolment, credit or certification. URLs below were accessed 17 August 2026.

MIT Missing Semester 2026. 2026 syllabus used for M0/M7.
https://missing.csail.mit.edu/

Stanford CS 190 Software Design Studio. Design pedagogy: decomposition, review and revision.
https://web.stanford.edu/~ouster/cs190-winter23/

John Ousterhout - A Philosophy of Software Design. Edition/source information; chapter mapping cross-checked against public contents/extract.
https://web.stanford.edu/~ouster/cgi-bin/aposd.php

Martin Fowler - Refactoring, 2nd ed.. Author overview; chapter contents cross-checked with O'Reilly/Pearson listing.
https://martinfowler.com/books/refactoring.html

Software Engineering at Google. Official free online table of contents.
https://abseil.io/resources/swe-book/html/toc.html

Python Testing with pytest, 2nd ed.. Publisher contents page.
https://pragprog.com/titles/bopytest2/python-testing-with-pytest-second-edition/

Introduction to Algorithms, 4th ed.. MIT Press edition information.
https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

Computer Systems: A Programmer's Perspective, 3rd ed.. Pearson table of contents; CMU CS:APP site for scope/labs.
https://www.pearson.com/en-us/subject-catalog/p/computer-systems-a-programmer-s-perspective/P200000003479/

CMU 15-213/15-513 Introduction to Computer Systems. Current course scope and resources.
https://www.cs.cmu.edu/~213/

CMU 15-445/645 Intro to Database Systems. Current course scope/projects; Fall 2026 landing page.
https://15445.courses.cs.cmu.edu/

Computer Networking: A Top-Down Approach, 9th ed.. Pearson table of contents.
https://www.pearson.com/en-us/subject-catalog/p/computer-networking-a-top-down-approach/P200000013385/

Designing Data-Intensive Applications, 2nd ed.. O'Reilly February 2026 edition and table of contents.
https://www.oreilly.com/library/view/designing-data-intensive-applications/9781098119058/

Fundamentals of Software Architecture, 2nd ed.. O'Reilly table of contents.
https://www.oreilly.com/library/view/fundamentals-of-software/9781098175504/

MIT 6.5840 Distributed Systems, Spring 2026. Current course structure, readings and labs.
https://pdos.csail.mit.edu/6.824/

OWASP Top 10:2025. Current awareness baseline.
https://owasp.org/Top10/2025/

OWASP ASVS 5.0. Application Security Verification Standard.
https://owasp.org/www-project-application-security-verification-standard/

Google Site Reliability Engineering. Official free online book.
https://sre.google/sre-book/table-of-contents/

Google Site Reliability Workbook. Official free online workbook.
https://sre.google/workbook/table-of-contents/

Design influences

- Stanford CS 190: iterative software design through implementation, review and revision.
- CMU 15-213: programmer-focused systems understanding as a base for operating systems, networking and architecture.
- CMU 15-445: project-heavy database internals rather than SQL syntax alone.
- MIT 6.5840: distributed systems taught through papers, discussion and substantial programming labs.
- MIT Missing Semester: practical tooling, debugging, packaging and professional workflow often omitted from conventional courses.
- Google engineering/SRE texts: code review, testing, CI/CD, reliability and operational practice as first-class engineering work.

Completion record

Use this final page as a compact evidence index. A future interviewer should be able to follow the artefacts and see not only what you built, but how your judgement developed.

Project I repository / evidence:

Project II deployment / repository:

Architecture portfolio / ADRs:

System-design practice evidence:

Capstone evidence:

- [ ] I can build and package maintainable software without relying on AI for basic implementation.
- [ ] I can design a test strategy and use failures as evidence rather than guesswork.
- [ ] I can reason from systems/database/networking models when diagnosing behaviour.
- [ ] I can deploy, observe, secure and recover a service.
- [ ] I can identify architectural drivers before choosing technologies.
- [ ] I can defend trade-offs and explicitly reject unnecessary complexity.
- [ ] I can reason about distributed failure and know when not to distribute.
- [ ] I can use AI aggressively while retaining ownership of specification, design and verification.

Final completion date:

Notes / evidence / reflection